Manager de portofoliu la GigiQuant

Alin-Teodor Bratu, Andra-Elena Băltoiu

Descrierea Proiectului

Proiectul conține 4 task-uri și un bonus structurate sub forma unor interviuri pentru poziția de manager de portofoliu la compania fictivă „GigiQuant”. Pentru a trece aceste interviuri, vei folosi structuri de date *(liste, stive, cozi, arbori binari, grafuri)* și vei dezvolta algoritmi pentru a rezolva diferite probleme din domeniul financiar *(calculul indicatorului Sharpe Ratio, identificarea oportunităților de arbitraj, optimizarea de portofoliu, determinarea probabilității ca o acțiune să aibă un preț după un număr de zile)*. Există recomandări de implementare pentru fiecare interviu, punctajul maxim fiind acordat doar dacă sunt respectate recomandările fiecărui interviu.

Ultimele 2 pagini conțin completări și exemple ale conceptelor prezentate în acest proiect. Nu sunt necesare rezolvării, doar clarifică câțiva din termeni folosiți.

Primul interviu - Sharpe ratio

Managerii de portofoliu au nevoie de metode de a-și evalua performanța. Cel mai simplu indicator care este cu adevărat relevant se numește **Sharpe Ratio**. Acesta arată cât de „profitabil” a fost riscul asumat de un portofoliu.

Unele investiții sunt mai sigure decât altele. Depozitul la bancă e considerat *risk free* pentru că la finalul anului primești, în plus, un procent mic din suma depozitată numit dobândă.

Spre deosebire de fizică unde randamentul este $\frac{\text{ieșire}}{\text{intrare}}$, în finanțe randamentul este $\frac{\text{profit}}{\text{investiție}}$ și adesea se calculează pe an. În cazul depozitului bancar, randamentul este egal cu dobânda.

Dacă banca îți oferă o dobândă garantată de **4%**, atunci portofoliul pe care îl construiești ar trebui:

- fie să ofere un randament mai mare de 4% cu certitudine mare (alfel ai lua cei 4% de la bancă),
- fie să ofere un randament considerabil mai mare (10–15%), dacă nu poate garanta siguranța investiției.

Un **fond de pensii**, de exemplu, este compus în principal din investiții sigure (titluri de stat, obligațiuni), deoarece scopul acestuia este să bată inflația și să-și mențină puterea de cumpărare, nu să-și îmbogățească proprietarii.

Un **hedge fund**, în schimb, este interesat de creșterea accelerată a portofoliului. Prin strategii mai riscante, reușește să obțină randamente mult mai mari.

Riscul, în acest context, reprezintă **volatilitatea portofoliului** — adică cât de mult fluctuează valoarea lui în timp.

Intuitiv:

- dacă graficul (Figura 1) seamănă cu o linie aproape dreaptă $\rightarrow$ risc mic (albastru)
- dacă graficul (Figura 1) seamănă cu un seismograf $\rightarrow$ risc mare (roșu)

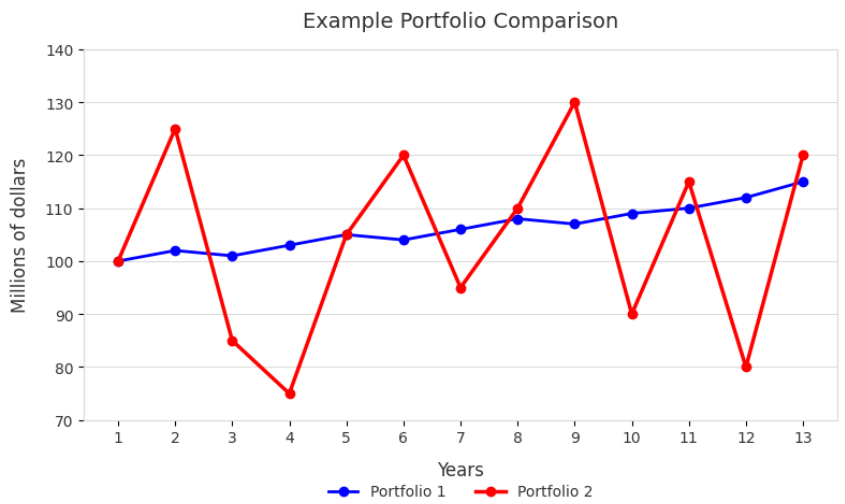


Figura 1: *Portofoliul albastru este considerat mai sigur decât cel roșu deoarece fluctuează mai puțin.*

Obiectiv:

În acest interviu va trebui să calculezi indicatorul Sharpe Ratio. Pentru aceasta, vei modela evoluția unui portofoliu pe mai multe zile folosind o listă, vei determina randamentul și volatilitatea acestuia (variabilele indicatorului), iar apoi vei calcula și afișa rezultatul final.

Formula pentru Sharpe Ratio este:

$$S = \frac{\mu - R_f}{\sigma}$$

unde:

- S = Sharpe Ratio
- μ = randamentul mediu al portofoliului

- R_f = randamentul fără risc
- σ = volatilitatea portofoliului (deviația standard a randamentelor)

Randamentul la un moment de timp t este definit ca:

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}} = \frac{\text{prețul la momentul } t - \text{prețul la momentul } (t-1)}{\text{prețul la momentul } (t-1)}$$

Randamentul mediu este:

$$\mu = \frac{1}{N}\sum_{t=1}^{N} R_t$$

Volatilitatea (deviația standard) este:

$$\sigma = \sqrt{\frac{1}{N}\sum_{t=1}^{N}(R_t - \mu)^2}$$

unde:

- P_t = valoarea portofoliului la momentul t
- N = numărul total de observații

Pentru a simplifica calculele considerăm $R_f = 0$.

Recomandari de implementare:

Folosește liste simplu înlănțuite. Listele vor fi folosite pentru a stoca valoarea portofoliului și randamentul acestuia.

Datele de intrare sunt secvențiale și calculul randamentul se bazează doar pe valoarea actuală și pe cea precedentă, așadar structura de date ce modelează cel mai bine problema este lista simplu înlănțuită. Un șir/array poate avea limitări de spațiu și structura nu se potrivește la fel de bine din punct de vedere logic (chiar dacă ai putea rezolva problema mai eficient cu un șir, este mult mai probabil să apară bug-uri logice).

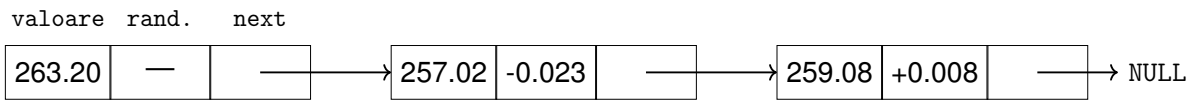
Un element din listă poate arăta în felul următor (poate fi numit și altfel):

```
struct Node {
    double valoare;
    double randament; // primul nod al listei evident nu v-a avea o valoare relevanta
        ↪ pentru asta
    struct Node* next;
};
```

Exemplu de input:

```
10          <- N (numărul total de observații)
263.20      <- P (valoarea acțiunii/portofoliului în ziua respectivă)
257.02
259.08
259.16
258.72
259.49
260.65
257.90
252.73
248.71
```

Exemplu reprezentare:



Exemplu de output:

```
-0.006  <- randamentul mediu
0.010   <- volatilitatea
-0.570  <- Sharpe ratio
```

Randamentul mediu poate fi calculat fie în timpul citirii inputului, fie printr-o parcurgere ulterioară a listei. Volatilitatea se obține reparcurgând lista (formula include randamentul mediu, deci nu poate fi calculată în parcurgerea inițială). Acești doi parametri și Sharpe Ratio trebuie stocați în fișierul de ieșire specificat ca argument.

Simplificări necesare: Prețurile au o acuratețe de 2 zecimale, însă variabilele din output vor avea o acuratețe de 3 zecimale. Rotunjirea se face prin trunchiere.

Al doilea interviu - Arbitraj

Informația este utilă numai dacă ești primul care are acces la ea. Cel puțin ăsta este cazul pentru arbitraj. Cam orice negustor vrea să facă arbitraj, să cumpere de undeva ieftin și să vândă scump altundeva (ca să facă profit). Paul Reuter a observat în secolul al 19-lea că legătura de informații dintre piața de la Paris și cea din Berlin era destul de lentă, ca urmare a antrenat porumbei ca să afle el primul ce se întâmplă în celălalt oraș. Dacă prețul unei acțiuni/unui bun creștea în celălalt oraș, urma să crească și în orașul în care se afla el (odată ce restul oamenilor aflau), deci cumpăra acțiuni la preț mic, aștepta câteva ore, apoi vindea totul pentru un profit aproape garantat. În ziua de azi diferențele ce aduc milioane durează milisecunde, nu ore.

La acest interviu va trebui să identifici și să capitalizezi oportunități de arbitraj. Pentru fiecare zi de tranzacționare, se cunosc prețurile aceleiași acțiuni pe trei piețe diferite, notate:

$$P_1,\ P_2,\ P_3$$

unde P_i este prețul acțiunii pe piața i.

Prețurile de pe cele 3 piețe se pot găsi în diverse relații. Pe baza stărilor posibile, GigiQuant a dezvoltat următoarea strategie de arbitraj:

- **Fără oportunitate de arbitraj:** dacă

  $$P_1 = P_2 = P_3$$

  Prețurile fiind identice pe toate piețele nu poți presupune mișcarea lor.

- **Oportunitate de arbitraj:** dacă exact două prețuri sunt egale și al treilea este diferit, adică:

  $$(P_1 = P_2 \neq P_3) \ \vee \ (P_1 = P_3 \neq P_2) \ \vee \ (P_2 = P_3 \neq P_1)$$

  Toată lumea vrea să facă arbitraj, deci diferențele o să fie minimizate/eliminate de toți investitorii de pe piață. În ipostaza asta vinzi acțiunea pe piața scumpă și o cumperi de pe piața ieftină, la fel ca Paul.

- **Situație de evitat:** dacă toate cele trei prețuri sunt distincte:

  $$P_1 \neq P_2 \neq P_3 \neq P_1$$

  Situația asta nu oferă informații prea clare. E posibil ca un eveniment global să fi destabilizat piața, caz în care arbitrajul devine riscant, deci nu are sens să fie făcut.

Obiectiv.

Se dă un fișier ce conține numele a 3 piețe și un număr necunoscut de prețuri în ordine cronologică pentru fiecare, adică primul preț e cel mai vechi disponibil și ultimul este cel mai nou. Este garantat că ultimul preț este din aceeași zi pentru toate fișierele și că nu există zile lipsă. Acest fișier cu prețuri trebuie prelucrat în așa fel încât să separați piețele și să indentificați oportunitățiile de arbitraj. Fișierul de output va conține ziua în care a existat oportunitatea, diferența absolută între valoarea diferită și celălalte două precum și numele acesteia.

Recomandari de implementare:

Folosește structura de date stivă (în exemplul din figura 2 am folosit 3 stive) pentru a stoca prețurile celor 3 piețe și o coadă pentru a stoca oportunitățiile de arbitraj (ziua în care are loc, diferența absolută între piețe și numele pieței diferită).

Exemplu de input:

Londra (P_1)	**Berlin (P_2)**	**Paris (P_3)**
160	230	140
120	120	130
120	213	123
123	120	
	123	

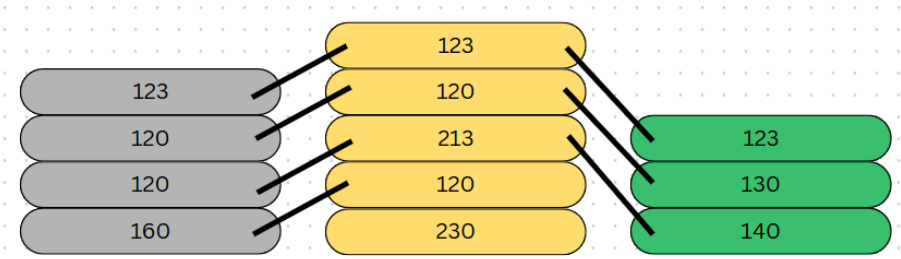


Figura 2: *Cele trei stive reprezintă datele despre aceeași acțiune de pe cele 3 piețe. Elementele de pe stivă legate cu linii oblice semnifică ziua. Cele 3 valori de 123 din vârful stivei au fost înregistrate în aceeași zi, nu există posibiliate de arbitraj. Ziua anterioară 120-120-130 prezintă o oportunitate de arbitraj. Ziua cu 120-213-140 nu reprezintă o oportunitate de arbitraj. Restul zilelor nu sunt luate în calcul pentru că stiva verde nu mai are elemente (acțiunea nu se tranzacționa pe piața aceea).*

Exemplu de output:

```
ziua 2 - 10 - Paris
```

Al treilea interviu - Diversificarea portofoliului

În primul interviu ai calculat Sharpe Ratio ca să poți distinge o acțiune riscantă de una sigură, **în acest interviu o să optimizezi un portofoliu** prin diversificare (adică transformi mai multe acțiuni riscante într-un portofoliu sigur). Diversificarea poate fi făcută fundamental prin analiza companiilor și investirea în domenii diferite de activitate, însă poate fi făcută și automat prin modelarea pieței cu arbori binari. Dacă fiecare zi în care o acțiune crește reprezintă o mișcare la dreapta, iar fiecare zi în care scade reprezintă o mișcare la stânga, atunci arborele generat de mai multe acțiuni ilustrează și o relație între acestea bazată pe strămoșii comuni. Figura 3 este un exemplu.

Se vor folosi simbolurile acțiunilor în loc de numele lor. AAPL = Apple, NVDA = Nvidia, etc. Simbolurile au un număr de caractere standard și conțin numai litere.

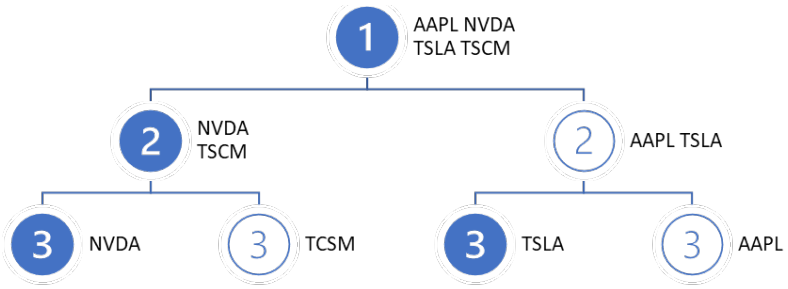


Figura 3: *Rădăcina (1) o considerăm neutră, este prima zi de analiză și nu știm cum s-a modificat prețul. În zilele ulterioare (2) și (3) acțiunile coboară pe ramurile adecvate.*

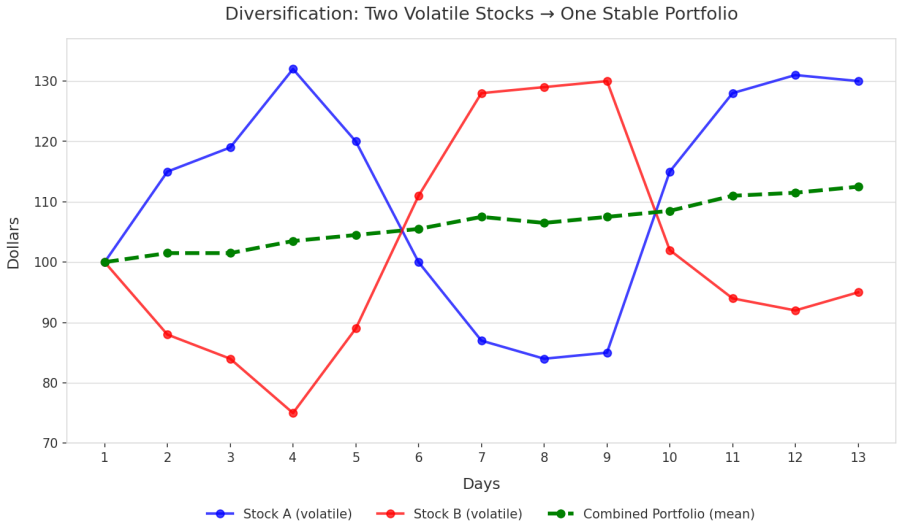


Figura 4: *Așa cum am discutat în primul interviu, un stock stabil n-ar trebui să varieze prea mult, chiar dacă variază în sus. Combinând două acțiuni în oglindă ca NVDA-AAPL sau TCSM-TSLA din figura 2, putem obține un portofoliu stabil.*

Figura 4 este un exemplu minimal de optimizare. În general portofoliile conțin mai mult de 2 acțiuni combinate în moduri sofisticate pentru a optimiza diverși indici. Dar asta este treaba unui manager de portofoliu. **La acest interviu o să primești 10 acțiuni și va trebui să le găsești opusul fiecăreia, adică oglinditul dintr-un arbore binar format după regulile precizate.**

Recomandari de implementare:

Trebuie să folosești un arbore binar care modelează mișcarea acțiunilor. Relațiile finale trebuie să provină din parcurgerea arborelui.

Exemplu de input:

AAPL,MSFT,AMZN,GOOG,META,TSLA,NVDA,INTC,ORCL,PYPL <- numele acțiunii
272.81,390.88,206.83,303.94,643.45,402.94,181.25,44.46,143.88,45.81
262.41,392.86,208.55,302.96,637.16,390.60,175.01,44.20,141.62,44.82
263.48,393.14,203.10,298.42,648.29,395.09,178.49,43.44,144.75,44.73
↑prețul acțiunii

Exemplu de output:

```
AAPL-AMZN      <- simbolurile acțiunilor opuse, în ordinea în care au fost date,
MSFT-GOOG       adică MSFT e în stânga lui GOOG pentru că a fost dat înaintea
MSFT-INTC       lui în listă, legătura MSFT-INTC e după MSFT-GOOG pentru că INTC
MSFT-PYPL       a fost dat mai târziu în listă decât GOOGLE (index 3 vs index 7)
AMZN-META
AMZN-TSLA
AMZN-NVDA
AMZN-ORCL
```

Un nod din arbore se poate implementa așa (dar ești liber să stochezi numele acțiunilor în orice structură de date):

```c
typedef struct StockList {
    char symbol[MAX_SYM];// NVDA are 4 caractere, poti face MAX_SYM = 5 pentru final \0
    struct StockList *next;
} StockList;

typedef struct TreeNode {
    StockList *stocks;      // toate actiunile care trec prin acest nod
    struct TreeNode *left;  // aici se duc actiunile care scad
    struct TreeNode *right; // aici se duc actiunile care cresc
    int depth;              // poate simplifica niste calcule
} TreeNode;
```

Al patrulea interviu - Lanțuri Markov

În realitate, prețurile nu sunt haotice complet. Deși pe termen scurt par aleatorii, pe termen mediu există **tipare de tranziție** între niveluri de preț. Un mod matematic de a modela acest comportament este folosind **lanțuri Markov**.

Ideea este simplă:

- împărțim axa prețurilor în **intervale fixe** (de exemplu: 100–101, 101–102, 102–103, etc.)
- fiecare interval devine o **stare**
- fiecare trecere de la o zi la alta reprezintă o **tranziție** între două stări

Astfel, construim o **matrice de tranziție** T (aproape identică cu matricea de adiacență), unde:

$$T_{ij} = P(\text{starea } i \to \text{starea } j)$$

adică probabilitatea ca prețul să treacă din intervalul i în intervalul j într-o zi. Această probabilitate se calculează în mod empiric, adică observi cât de des trece prețul din starea i în starea j raportat la cât de des trece din starea i la orice altă stare înafară de starea j.

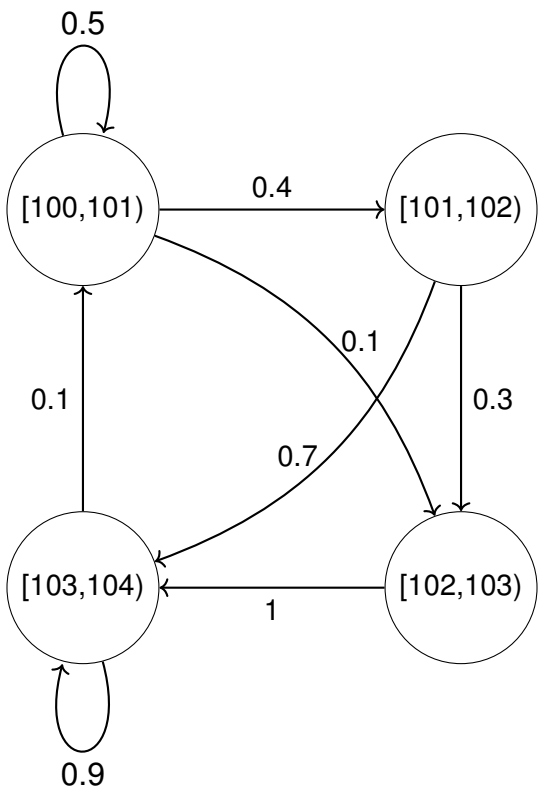


Figura 5: *Graf Markov cu intervale de preț și probabilități de tranziție*

Input:
Fișierul de intrare conține:

- o serie de prețuri zilnice $P_1, P_2, \ldots, P_N$
- dimensiunea intervalului Δ (de exemplu $\Delta = 1$ dolar)
- un preț inițial P_{start}
- un preț țintă P_{target}
- un număr de zile K

Obiectiv:

1. Construiește intervalele de preț (stările Markov)
2. Construiește matricea de tranziție T sau graful cu muchii și noduri
3. Calculează probabilitatea ca acțiunea să ajungă din intervalul lui P_{start} în intervalul lui P_{target} în fiecare zi timp de K zile. **Probabilitatea trebuie scrisă sub formă de fracție ireductibilă. Probabilitatea trebuie determinată algoritmic (simulând trecerea zilelor pe graf), nu algebric (prin ridicarea la putere a matricei de tranziție).**

Exemplu de input:

```
10              <- N (număr observații)
1.0             <- d (dimensiunea intervalului)
5               <- K (număr de zile)
100.0           <- P_start
101.0           <- P_target
100.2
101.1
101.9
102.4
101.7
103.2
104.1
104.9
105.3
104.8
```

Exemplu de output:

```
0       <- probabilitatea ca prețul să fie P_target (101) în ziua 1
1
1/3
4/9
7/27  <- probabilitatea ca prețul să fie P_target în ziua K
```

Figura 6 arată în stânga graful după împărțirea valorilor în intervale și în partea dreaptă graful cu muchiile reprezentând probabilitatea de tranziție dintr-o stare în alta. Probabilitatea de tranziție se determină prin raportul dintre numărului de săgeți care ies dintr-o stare i și câte dintre ele se duc în starea j. Figurile 7- 11 arată evoluția stării prețului pentru acest lanț Markov care începe de la starea 100.

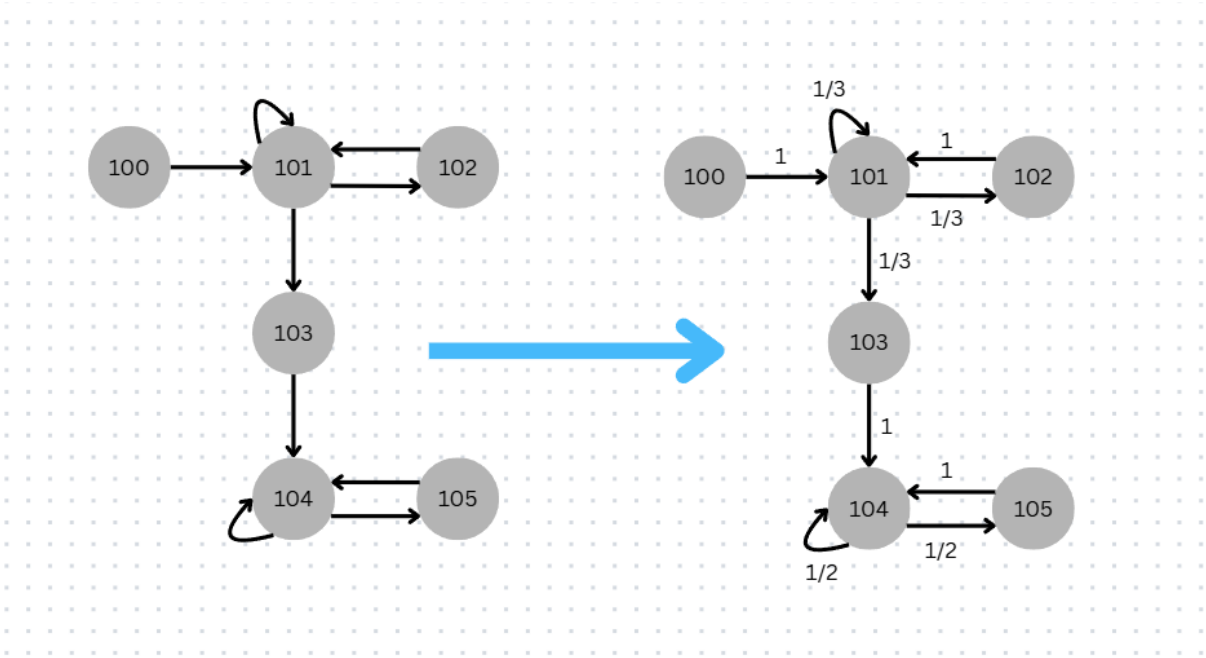


Figura 6: Transformarea unei graf orientat într-un lanț Markov

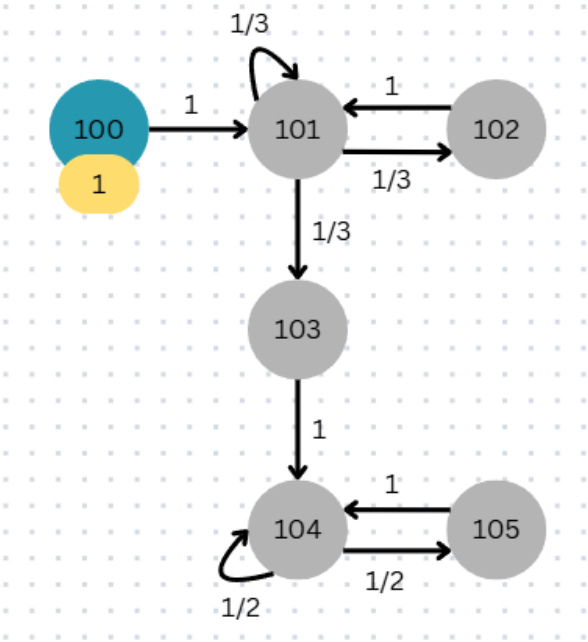


Figura 7: În prima zi, prețul se află în starea 100, adică aparține intervalului [100,101). Culoarea albastră a cercului arată că starea este „activă”, adică ne aflăm în ea și putem merge în altă stare ziua următoare. Ovalul galben indică probabilitatea să ne aflăm în acea stare, în prima zi probabilitatea este mereu 1. Din starea asta, prețul se poate duce numai în starea 101.

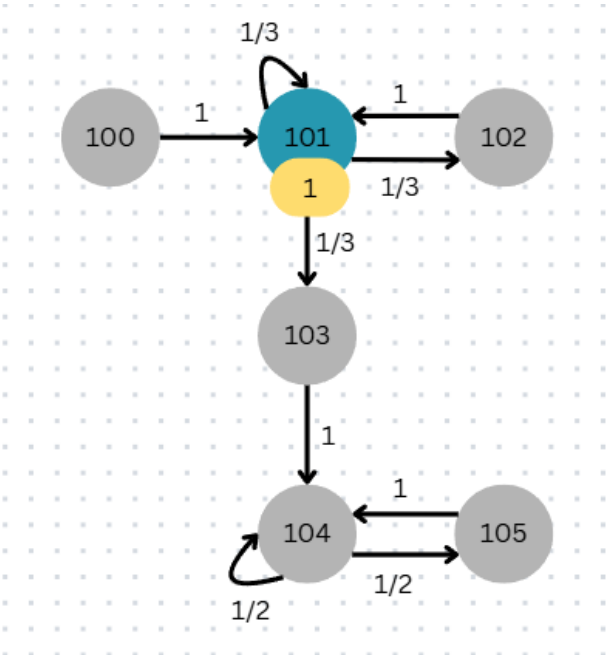


Figura 8: Starea prețului s-a schimbat din 100 în 101. De aici prețul poate să rămână în 101 cu probabiliatea de $\frac{1}{3}$, să se ducă în 102 cu probabiliatea de $\frac{1}{3}$ sau în 103 cu aceeași probabilitatea. Următoarea zi, toate cele 3 opțiuni vor fi active.

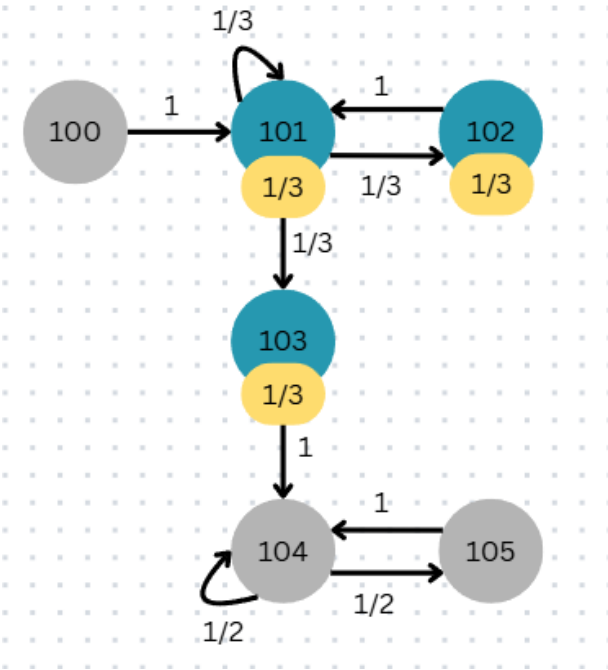


Figura 9: 3 stări sunt active (101, 102 și 103). Fiecare va fi tratată pe rând, apoi dacă 2 (sau mai multe) duc în aceeași stare, probabilitățiile se for aduna.

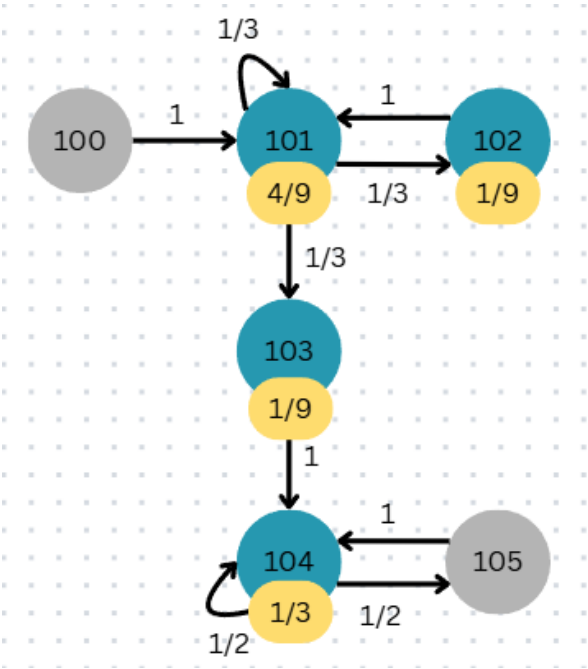


Figura 10: Probabiliatea de 4/9 a stării 101 provine din două stări precedente (101 și 102). 101 are probabilitea de 1/3 să se reîntoarcă în 101 și prețul avea probabilitea de 1/3 să se afle în starea 101, deci 1/3 * 1/3 = 1/9. 102 are probabilitatea de 1 să se ducă în 101 și probabilitatea prețului de a fi în starea 102 era de 1/3, deci 1/3 * 1 = 1/3. Suma celor 2 tranziții este 1/9 + 1/3 = 4/9.

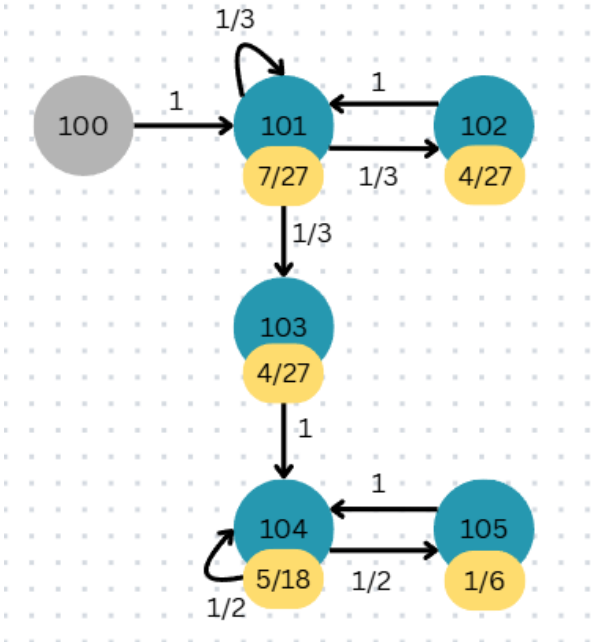


Figura 11: Francțiile au fost duse într-o formă ireductibilă.

Interviu bonus - folosirea unui API

Toate datele folosite de-a lungul interviurilor au fost obținute prin API-ul de la yahoo finance care este complet gratuit și foarte ușor de folosit în python. Ca să arăți totuși că ești cel mai apt candidat pentru GigiQuant, îți vei obține datele prin cod C. Pentru a simplifica anumite aspecte, poți descărca fișierele cJson.c și cJson.h de pe github din proiectul cJson.

Astea-s bibliotecile de care ai nevoie:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <curl/curl.h>
#include "cJSON.h"
```

Astea-s funcțiile de care ai nevoie:

```c
struct MemoryStruct {
    char *memory;
    size_t size;
};

static size_t WriteMemoryCallback(void *contents, size_t size, size_t nmemb, void *
    ↪ userp) {
    size_t realsize = size * nmemb;
    struct MemoryStruct *mem = (struct MemoryStruct *)userp;
    char *ptr = realloc(mem->memory, mem->size + realsize + 1);
    if (!ptr) return 0;
    mem->memory = ptr;
    memcpy(&(mem->memory[mem->size]), contents, realsize);
    mem->size += realsize;
    mem->memory[mem->size] = 0;
    return realsize;
}

double* get_open_prices(const char* symbol, const char* interval, const char* range,
    ↪ int *count_out) {
    CURL *curl_handle;
    struct MemoryStruct chunk = {malloc(1), 0};
    char url[256];
    double *prices = NULL;

    sprintf(url, "https://query1.finance.yahoo.com/v8/finance/chart/%s?interval=%s&
        ↪ range=%s", symbol, interval, range);
    curl_handle = curl_easy_init();

    curl_easy_setopt(curl_handle, CURLOPT_URL, url);
```

```
    curl_easy_setopt(curl_handle, CURLOPT_WRITEFUNCTION, WriteMemoryCallback);
    curl_easy_setopt(curl_handle, CURLOPT_WRITEDATA, (void *)&chunk);
    curl_easy_setopt(curl_handle, CURLOPT_USERAGENT, "libcurl-agent/1.0");

    if (curl_easy_perform(curl_handle) == CURLE_OK) {
        cJSON *json = cJSON_Parse(chunk.memory);
        cJSON *result = cJSON_GetArrayItem(cJSON_GetObjectItem(cJSON_GetObjectItem(json
            ↪ , "chart"), "result"), 0);
        cJSON *quote = cJSON_GetArrayItem(cJSON_GetObjectItem(cJSON_GetObjectItem(
            ↪ result, "indicators"), "quote"), 0);
        cJSON *open_prices = cJSON_GetObjectItem(quote, "open");

        *count_out = cJSON_GetArraySize(open_prices);
        prices = malloc((*count_out) * sizeof(double));

        for (int i = 0; i < *count_out; i++) {
            cJSON *val = cJSON_GetArrayItem(open_prices, i);
            prices[i] = cJSON_IsNumber(val) ? val->valuedouble : -1.0;
        }
        cJSON_Delete(json);
    }

    curl_easy_cleanup(curl_handle);
    free(chunk.memory);
    return prices;
}
```

Apelând funcția `get_open_prices()` obții prețul unei acțiunii la deschiderea pieței pe un interval de zile. **La acest interviu trebui doar să arăți că știi să chemi API-ul și să faci ceva (inclusiv ce ai făcut la interviurile precedente) cu datele returnate**.

Informații suplimentare despre domeniu

Această secțiune este dedicată celor interesați de domeniul financiar sau celor ce ar prefera mai multe clarificări în legătură cu conceptele folosite în temă.

S-ar putea să pară ciudat să numești dobândă atât banii pe care ți dă banca când ai un depozit, cât și banii pe care trebuie să îi dai băncii când ai un împrumut, însă cele 2 situații sunt, de fapt, aceeași chestie. Banca nu-ți dă împrumut din banii ei, atunci când îți depozitezi banii la bancă imiplicit îi permiți să-i folosească cum vrea ea atâta timp cât ți poate returna la nevoie, deci împrumutul provine din depozitul cuiva. Poți considera banca ca un intermediar între oamenii cu exces de fonduri și cei cu nevoie de fonduri. Sursa principală de venit a unei bănci tradiționale este dobânda provenită din credite, o parte din dobânda provenită din credite se va întoarce către oamenii cu depozite la banca (care practic finanțează creditele altora), deci fiind aceeași sumă de bani are și același nume. Banca Națională sau alte instutuții în funcție de țară reglementează câți bani trebuie să aibe banca în orice moment. În general rezerva obligatorie este de 10-15% din valoarea tuturor depozitelor, asta înseamnă că 90% pot fi dați ca împrumut. Din acest motiv dobânda unui credit este mai mare decât a unui depozit.

Niște instrumente financiare similare dobânzii bancare sunt obligațiuniile și titlurile de stat. Obligațiunile sunt un fel de împrumuturi pe care le faci unei companii, „dobânda” lor se numește „cupon” și în general este mai mare decât ce-ți oferă banca. Obligațiunile sunt considerate sigure, însă dacă compania dă faliment nu-ți vei mai putea recupera banii. Titlurile de stat sunt împrumuturi pentru state. În mod evident titlurile de stat sunt cele mai sigure investiții, singurul caz în care îți poți pierde banii este dacă țara dă faliment, asta se întâmplă foarte rar. În România poți primi 1% suplimentar pentru titlurile de stat dacă ești donator de sânge. Banca în general nu-ți asigură întreg depozitul. În 2022, Silicon Valley Bank a dat faliment, foarte multe company și start-up-uri din tech aveau conturi acolo, banca asigura numai sume până la 250k dolari, evident multe depozite erau mai mari de atât.

Sharpe Ratio este un indicat clasic, dar nu-i cel mai bun pentru că vede volatilitatea ca fiind rea chiar dacă portofoliul crește și pentru că se raportează la Risk Free Rate în loc să se raporteze la restul pieței. Information Ratio rezolvă unele din problemele menționate, dar nici el nu e perfect și uneori poate face un portofoliu bun să pară slab. Ideal este să folosești mai mulți indicatori pentru o analiză completă.

Uneori diversificarea este mai puțin intuitivă. De exemplu, o companie de asigurări vinde „abonamente” pe o durată fixă de timp, deci crizele financiare tind să-o afecteze mai târziu. Există o companie română de investiții ce deține una dintre cele mai mari platforme de pariuri (care ar fi afectate imediat în cazul unei crize financiare) și o firmă de asigurări de mașină (care ar fi afectată după un an).

Arbitrajul Statistic este varianta mai complicată a Pairs Trading, tema în realitate e mai aproape de pairs decât de arbitrajul modern. Conceptul principal pentru ambele este mean reversion sau revenirea la medie, adică acțiunile au un comportament normal la care se întorc.

Abordarea diversificării portofoliului prin arbori binari are câteva probleme. Există câteva zeci de mii de acțiuni pe piață (majoritatea în SUA și China hartă). Arborele binar din temă își dublează numărul de frunze în fiecare zi , deci în mai puțin de o lună vor fi atât de multe frunze încât este puțin probabil să găsești o pereche perfect opusă. Metodele moderne se axează pe matrici de corelație care sunt mai permisive, dacă acțiunile se mișcă în aceași direcție într-o zi și în rest în direcții opuse, atunci tot o să fie corelate foarte negativ.

Dacă în loc să verifici oglinditul unei frunze ai verifica vecinii frunzei, atunci ai obține așa numitele sister stocks. Acestea sunt acțiuni foarte similare (cum ar fi CocaCola și Pepsi) care se mișcă cam la fel. Sister Stocks sunt foarte utile pentru pairs trading.

Lanțurile Markov sunt folosite în general la nivel macro, adică nu analizezi o acțiune de pe o zi pe alta, în schimb poate analizezi un portofoliu din 6 în 6 luni pentru a vedea efectul unor factori economico-sociali asupra lui.

Options este un termen care apare peste tot și poate să pară complicat, dar conceptul din spate este simplu. Practic o opțiune este un contract prin care un cumpărător își asigură dreptul să plătească prețul pieței de azi pentru un bun deținut de vânzător, dar achiziția să se facă (dacă cumpărătorul își mai dorește) în viitor. Pentru acest privilegiu, cumpărătorul plătește o mică taxă vânzătorului. Există 2 tipuri principale Americane și Europene. Primul contract de options a fost făcut de Thales care a cumpărat dreptul să închirieze prese de măsline înainte de recoltă la un preț mic, apoi în timpul recoltei le-a închiriat altor oameni cu un preț mult mai mare. În România între anii 2019 și 2023 a existat un fel de options trading în piața imobiliarelor. Dezvoltatorii de imobiliarea solicitau un avans (10-15% din valoarea finală) sau o rezervare (de vreo 1000 de euro) pentru a oferi cumpărătorului dreptul de a cumpăra un imobil la prețul „de proiect” sau „de groapă” care uneori poate să fie și jumătate din valoarea imobilului construit și mobilat. În perioda respectivă, clauzele contractului nu necesitau ca plătitorul avansului să fie și cumpărătorul imobilului, asta oferea oportunitatea de tranzacționare a contractului. Investitorii au profitat de asta în felul următor: Au plătit avansul, au așteptat până când dezvoltatorul a crescut prețul, apoi au vândut contractul la valoarea avansului + diferența dintre prețul din contract și prețul la zi al imobilului. Azi, astfel de investiții încă sunt posibile, însă majoritatea dezvoltatorilor au inclus clauze în contract ca să limiteze astfel de vânzări pentru a-și crește profitabilitatea.